



ELEVANCESKILL INTERNSHIP
ON
INTELLIGENT EV BATTERY MANAGEMENT SYSTEM

A Real-Time Multi-Cell Battery Intelligence,
Safety, Fault-Tolerant and Cloud Telemetry System.

[BMS / EV ICON]

SUBMITTED BY,

VIVEK BADGUJAR

B.Tech – Electronics & Telecommunication Engineering
MIT Academy of Engineering, Alandi

INTERNSHIP ORGANIZATION

ELEVANCESKILL

DOMAIN

Electrics Vehicle – Battery Management System

UNDER THE GUIDANCE OF

Mentor: Mr. Jayanth Sir

Date

20/08/2026

Offer Letter / Internship Selection Letter

ELEVANCE
SKILLS

elevanceskills.com 

support@elevanceskills.com 

+91 9134772858 

2/180, PUNGAMPATTI VILLAGE, BARUR POST,
POCHAMPALLI TALUK, KRISHNAGIRI, 635201

June 22, 2026

Dear Vivek Badgujar,

We are pleased to offer you the opportunity to join ElevanceSkills as an **Embedded Systems** Intern.

This offer is conditional upon Annexure A: Terms and Conditions attached below and your successful completion of the required training program. Upon fulfilling the training criteria, you will embark on a journey of professional growth and real-world experience with us.

Congratulations!



Jay Kumar,
Chief Human Resources Officer

CERTIFICATE



Declaration

I hereby declare that the internship project report entitled “**Intelligent EV Battery Management System**” is an original work carried out by me during my internship at **ElevanceSkills** under the guidance of the assigned mentor.

This project has been developed as an integrated **Electric Vehicle Battery Management System (EV-BMS)** using **ESP32, Wokwi, Arduino C++, and Blynk IoT**. The work presented in this report covers the implementation, integration, testing, and analysis of the six internship tasks assigned as part of the internship program.

I further declare that the information presented in this report is based on the work carried out during the internship and has not been submitted previously, in whole or in part, for the award of any other academic qualification or certification.

Name: Vivek Badgujar

Branch: Electronics & Telecommunication Engineering

College: MIT Academy of Engineering, Alandi, Pune

Place: Alandi, Pune

Date: 20/08/2026

Signature of Student

Vivek Ravindra Badgujar

Acknowledgement

I would like to express my sincere gratitude to **ElevanceSkills** for providing me with the opportunity to undertake this internship in the field of **Electric Vehicle (EV) Battery Management Systems**.

I am thankful to my internship mentor for providing continuous guidance, technical support, and valuable suggestions throughout the development of the project. The mentor's feedback helped me understand the practical aspects of battery monitoring, safety protection, fault-tolerant embedded systems, cloud telemetry, and IoT-based dashboard development.

I would also like to express my sincere gratitude to the faculty members and management of **MIT Academy of Engineering, Alandi, Pune**, for their support and encouragement during the internship.

I am thankful to everyone who directly or indirectly contributed to the successful completion of this project. This internship provided me with valuable practical experience in **ESP32, embedded C/C++, Wokwi simulation, Blynk IoT, battery intelligence, event-driven programming, fault handling, and embedded system development**.

Abstract

The increasing adoption of electric vehicles (EVs) requires reliable and intelligent Battery Management Systems (BMS) for continuous monitoring, safety protection, fault detection, and battery condition assessment. This internship project presents the design and implementation of an **Intelligent EV Battery Management System** developed as a single integrated embedded platform using **ESP32, Wokwi Simulation, and Blynk IoT**.

The system is designed to monitor a simulated **four-cell lithium battery pack** in real time. It acquires individual cell voltage data and calculates important battery parameters such as pack voltage, average cell voltage, voltage imbalance, and the strongest and weakest cells. Based on these parameters, the system classifies the battery condition into different health states.

An event-driven safety protection mechanism is implemented using a non-blocking millis()-based architecture. The system detects conditions including under-voltage, over-voltage, sensor anomalies, and rapid voltage changes, and responds through relay protection, buzzer alerts, and LCD warning messages. A fault-tolerant runtime subsystem further detects conditions such as invalid sensor readings, frozen ADC behaviour, and relay mismatch while managing different operational states including **NORMAL, DEGRADED, FAILSAFE, and SHUTDOWN**.

The project also incorporates an embedded LCD-based Human-Machine Interface (HMI) with automatic diagnostic page rotation and priority-based fault display. Fault events are timestamped and logged for diagnostic purposes. For remote monitoring, an event-driven cloud telemetry architecture using **Blynk IoT** transmits important data during state changes, anomalies, and significant voltage variations while supporting network reconnection and event synchronization.

Finally, an executive battery intelligence dashboard provides live battery information, historical voltage trends, risk and diagnostic status, fault history, and operator recommendations. The complete system demonstrates the integration of **battery intelligence, safety protection, fault tolerance, embedded HMI, cloud telemetry, and IoT-based visualization** into a single simulated EV-BMS platform.

Table of Contents

Sr. No.		Content	Page No.
		Offer Letter / Internship Selection Letter	1
		Certificate	2
		Declaration	3
		Acknowledgement	4
		Abstract	5
1		SYSTEM OVERVIEW	
	1.1	Introduction to Intelligent EV Battery Management System	9
	1.2	Problem Statement	9
	1.2	Project Objectives	9
	1.4	Scope of the Project	10
	1.5	System Components and Technologies	10
	1.6	System Architecture Diagram	11
	1.7	Architecture Description	11
	1.8	System Workflow Diagram	12
2		TASK 1 – Battery Health Monitoring System (4-Cell pack)	
	2.1	Task 1 Problem Statement	13
	2.2	Components Used	13
	2.3	Circuit Diagram / Connections	13
	2.4	Simulation Circuit	14
	2.5	Imbalance Calculation formulas	14
	2.6	Strongest and Weakest Cell Identification	15
	2.7	Battery Health Classification / Simulation Running Photos with correct Output:	15
	2.8	Important Notes:	16
	2.9	Task 1 Wokwi Link	17
	2.10	Task 1 Video Demonstration Link	17
3		TASK 2 – EVENT-DRIVEN SAFETY PROTECTION KERNEL	

	3.1	Task 2 Problem Statement	17
	3.2	Components	17
	3.3	Simulation Circuit	18
	3.4	Simulation Running Photos with correct Output	18
	3.5	Important Notes	19
	3.6	Task 2 Wokwi Link	19
	3.7	Task 2 Video Demonstration Link	19
4		TASK 3 – INTELLIGENT EMBEDDED HMI & DIAGNOSTIC INTERFACE	
	4.1	Task 3 Problem Statement	20
	4.2	Simulation Running Photos with correct Output	20
	4.3	Task 3 Wokwi Link	21
	4.4	Task 3 Video Demonstration Link	21
5		TASK 4 – FAULT-TOLERANT EMBEDDED RUNTIME SYSTEM	
	5.1	Task 4 Problem Statement	22
	5.2	Simulation Running Photos with correct Output	22
	5.3	Task 4 Wokwi Link	23
	5.4	Task 4 Video Demonstration Link	23
6		TASK 5 – INTELLIGENT CLOUD TELEMETRY ARCHITECTURE	
	6.1	Task 5 Problem Statement	24
	6.2	Blynk IoT Architecture	24
	6.3	Task 5 Wokwi Link	31
	6.4	Task 5 Blynk Dashboard Link	31
	6.5	Task 5 Video Demonstration Link	31
7		TASK 6 – EXECUTIVE BATTERY INTELLIGENCE DASHBOARD	
	7.1	Task 6 Problem Statement	32
	7.2	Final Integrated Circuit	32
	7.3	Blynk Dashboard Overview photos	32
	7.4	Final Wokwi Simulation Link	33
	7.5	Blynk Dashboard Link	33

	7.6	Task 6 Video Demonstration Link	34
8		Limitations	35
9		Conclusion	35
10		Appendix	35
11		References	36

1. SYSTEM OVERVIEW

1.1 Introduction to the Intelligent EV Battery Management System

The increasing adoption of Electric Vehicles (EVs) requires reliable Battery Management Systems (BMS) for continuous battery monitoring, safety protection, fault detection, and operational analysis. This project presents an **Intelligent EV Battery Management System** developed as a single integrated embedded system using an **ESP32 microcontroller, Wokwi simulation, and Blynk IoT**.

The developed system monitors a simulated four-cell lithium battery pack in real time. It measures individual cell voltages and calculates important battery parameters such as pack voltage, average cell voltage, voltage imbalance, and the strongest and weakest cells. Based on the calculated parameters, the system determines the battery health condition.

The system integrates six major functions: **Adaptive Multi-Cell Battery Intelligence, Event-Driven Safety Protection, Intelligent Embedded HMI, Fault-Tolerant Runtime Management, Intelligent Cloud Telemetry, and Executive Battery Intelligence Dashboard**. These functions operate together as a single integrated EV-BMS platform.

1.2 Problem Statement

Electric vehicle battery packs require continuous monitoring to ensure safe and reliable operation of individual cells. Abnormal voltage conditions, sensor faults, and system failures can affect battery performance and safety. This project develops an integrated EV-BMS for battery monitoring, safety protection, fault handling, cloud telemetry, and dashboard-based diagnostics.

1.3 Project Objectives

- i. **To develop a real-time four-cell battery intelligence system** for measuring cell voltages, pack voltage, average voltage, imbalance, and identifying the strongest and weakest cells.
- ii. **To classify battery health and implement safety protection** by detecting under-voltage, over-voltage, rapid voltage changes, and sensor anomalies using a non-blocking millis()-based architecture.

- iii. **To develop an intelligent embedded HMI and fault-tolerant runtime system** for LCD diagnostics, protection status, fault detection, runtime modes, fault logging, and recovery.
- iv. **To implement intelligent cloud telemetry using Blynk IoT** for event-based battery data transmission, Wi-Fi reconnection, and communication monitoring.
- v. **To develop an executive battery intelligence dashboard** providing live monitoring, risk analysis, historical voltage trends, diagnostics, fault history, and operator recommendations.

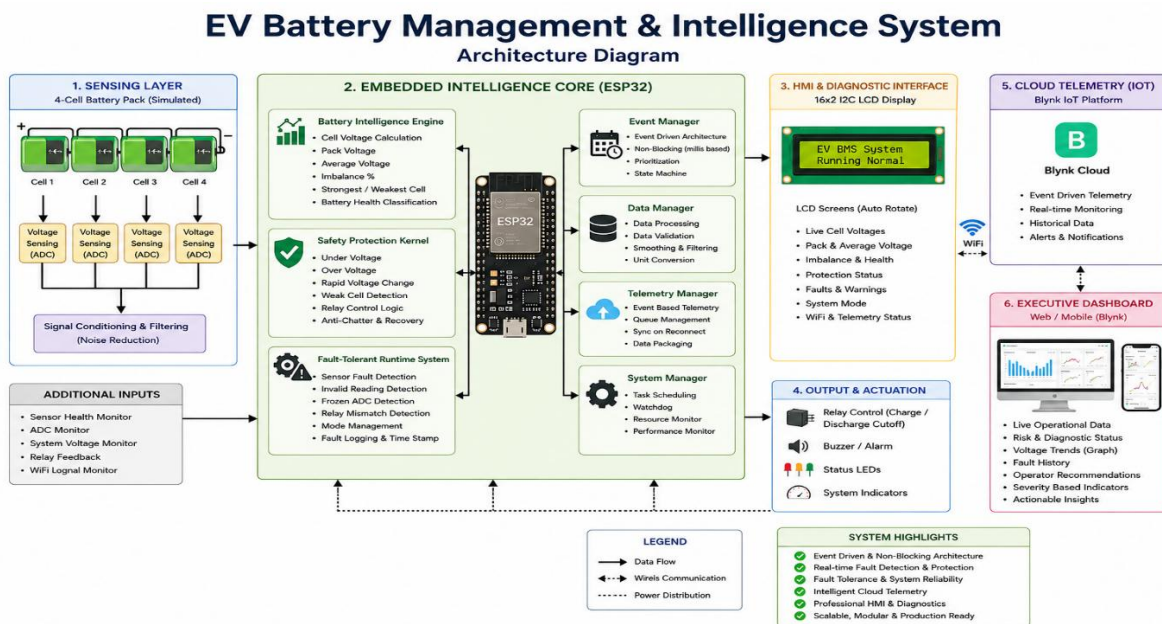
1.4 Scope of the Project

The project focuses on developing a **four-cell EV-BMS using ESP32 and Wokwi** for battery monitoring, safety protection, fault management, LCD diagnostics, Blynk cloud telemetry, and dashboard visualization. All six internship tasks are integrated into a **single system** for real-time simulation and validation.

1.5 System Components and Technologies

Category	Component / Technology
Microcontroller	ESP32
Battery	4-Cell Lithium Battery (For Simulation Potentiometer)
Display	16×2 I2C LCD
Protection	Relay & Buzzer
Indicators	LEDs
Programming	Arduino C++
Simulation	Wokwi
Cloud	Blynk IoT
Communication	Wi-Fi
Timing	millis()-based architecture

1.6 System Architecture Diagram



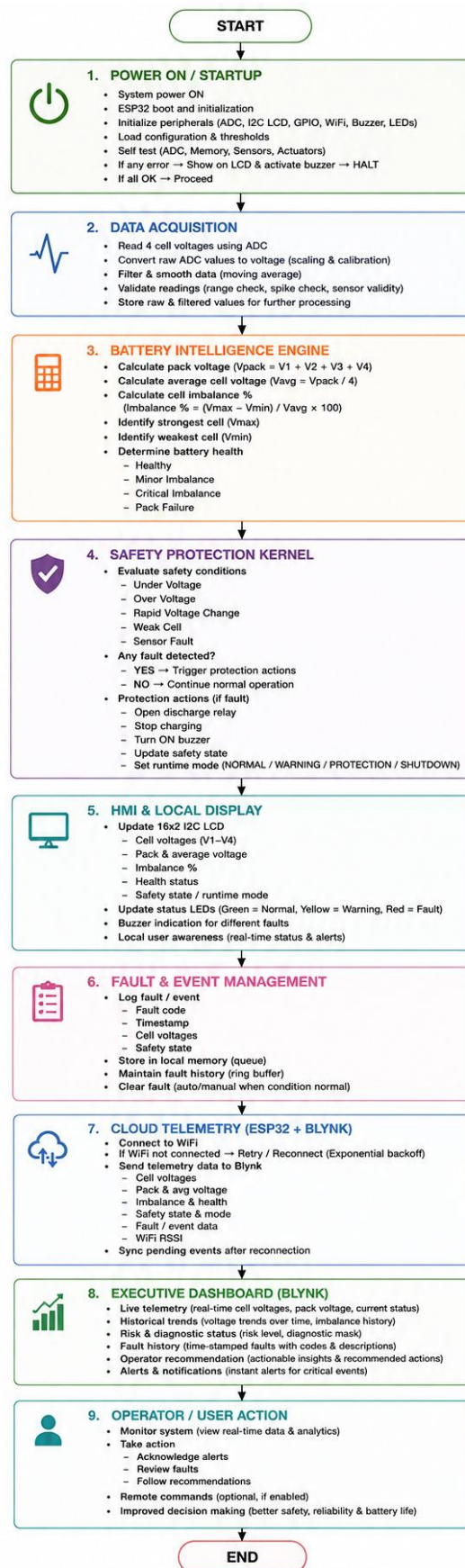
1.7 Architecture Description

The proposed EV-BMS architecture integrates **battery sensing, ESP32-based intelligence, safety protection, fault management, HMI, cloud telemetry, and dashboard monitoring** into a single system.

The **four-cell battery pack** provides individual cell voltage inputs to the sensing stage. The ESP32 processes these inputs to calculate battery parameters, detect abnormal conditions, manage faults, and control the protection outputs.

The processed information is displayed locally through the **16x2 I2C LCD** and transmitted through **Wi-Fi to the Blynk Cloud** for remote monitoring. The Blynk dashboard provides live battery data, risk status, historical trends, fault history, and operator recommendations.

1.8 System Workflow Diagram



2. Task 1: Battery Health Monitoring System (4-Cell pack)

2.1 Task 1 Problem

Adaptive Multi-Cell Battery Intelligence Engine: In this task, candidates are required to design and implement a production-grade battery intelligence engine capable of monitoring a simulated 4-cell lithium battery pack in real time. The system must read multiple analog inputs, calculate individual cell voltages, pack averages, imbalance percentages, and identify the weakest and strongest cells. It should also classify battery health into different states such as Healthy, Minor Imbalance, Critical Imbalance, and Pack Failure using scalable and modular embedded architecture principles.

2.2 Components:

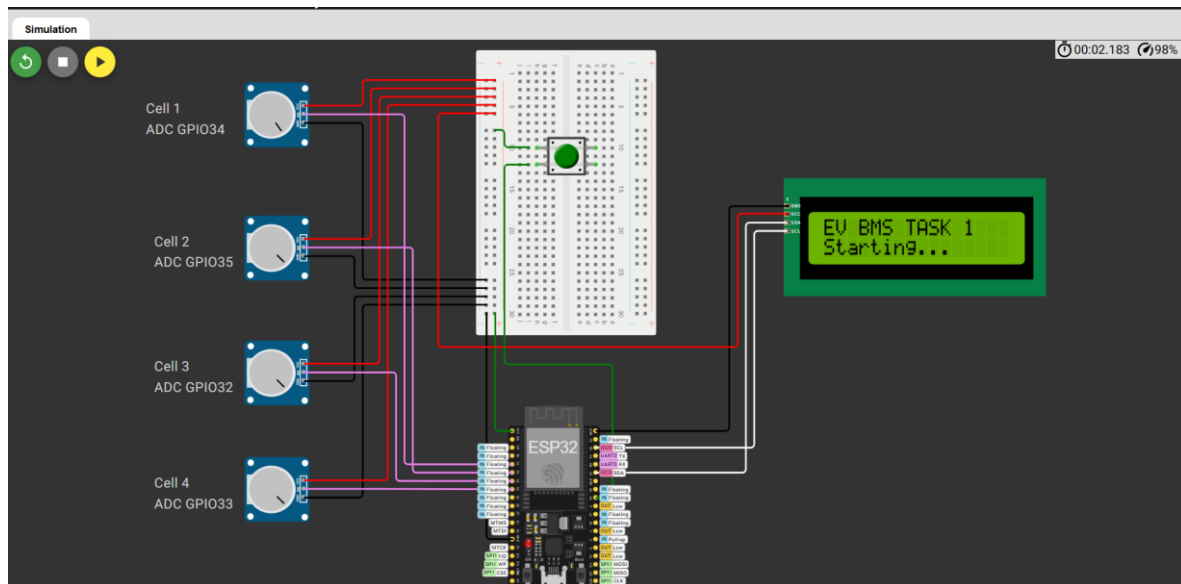
1. Microcontroller (ESP32- DevKit)
2. Breadboard
3. Pack of Cell (4-Cell) / Potentiometer (10K)
4. LCD 20×4 I²C
5. Push Button

2.3 Circuit Diagram Connection:

Component	Pin/Terminal	ESP32 Connection
Cell 1 Potentiometer	SIG	GPIO 34 (ADC)
	VCC	3.3 V
	Ground (GND)	GND
Cell 2 Potentiometer	SIG	GPIO 35 (ADC)
	VCC	3.3 V
	Ground (GND)	GND
Cell 3 Potentiometer	SIG	GPIO 32 (ADC)
	VCC	3.3 V
	Ground (GND)	GND
	SIG	GPIO 33 (ADC)
	VCC	3.3 V

Cell 4 Potentiometer	Ground (GND)	GND
16×2 I ² C LCD	SDA	GPIO 21
	SCL	GPIO 22
	VCC	3.3 V
	GND	GND

2.4 Simulation Circuit :



2.5 Imbalance Calculation formulas

a. Imbalance Voltage:

$$\Delta V = V_{\max} - V_{\min}$$

b. Imbalance Percentage:

$$\text{Imbalance\%} = [(V_{\max} - V_{\min}) / V_{\text{avg}}] \times 100$$

Example

If: Cell 1 = 3.70 V
 Cell 2 = 3.68 V
 Cell 3 = 3.72 V
 Cell 4 = 3.65 V

Then:

Pack Voltage : 3.70+3.68+3.72+3.65=14.75V

Average Voltage : 14.75/4=3.6875V

2.6 Strongest & Weakest Cell:

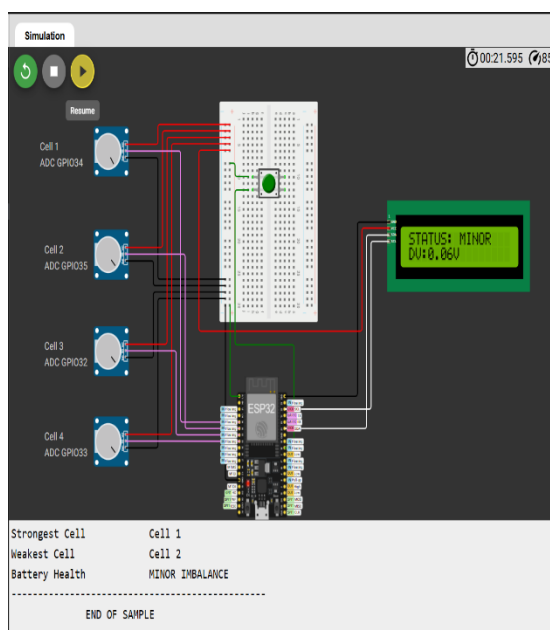
Strongest Cell: 3.72 V → Cell 3

Weakest Cell: 3.65 V → Cell 4

Imbalance : 3.6875
 $(3.72 - 3.65) \times 100 \approx 1.90\%$

2.7 Simulation Running Photos with correct Output:

Battery Health = Minor Imbalance



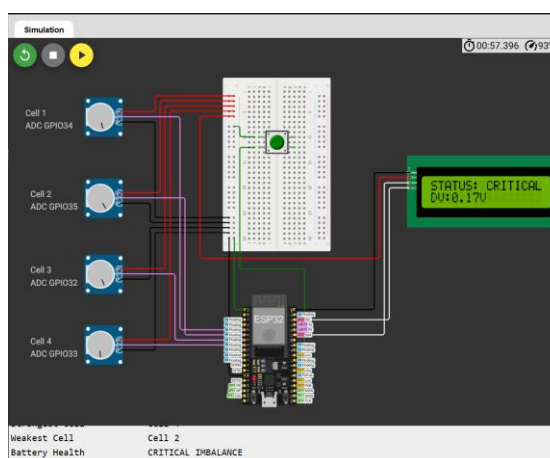
System started successfully.

ADAPTIVE BATTERY INTELLIGENCE

Parameter	Value
Cell 1 Voltage	2.56 V
Cell 2 Voltage	2.50 V
Cell 3 Voltage	2.50 V
Cell 4 Voltage	2.50 V
Pack Voltage	10.06 V
Average Voltage	2.52 V
Maximum Voltage	2.56 V
Minimum Voltage	2.50 V
Imbalance Voltage	0.06 V
Imbalance Percentage	2.51 %
Strongest Cell	Cell 1
Weakest Cell	Cell 2
Battery Health	MINOR IMBALANCE

END OF SAMPLE

Battery Health = Critical Imbalance

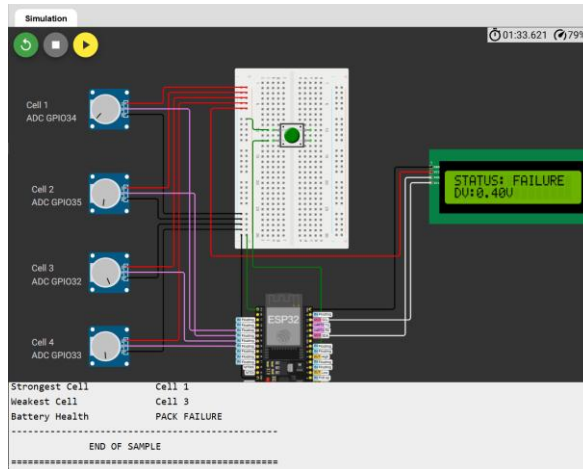


ADAPTIVE BATTERY INTELLIGENCE

Parameter	Value
Cell 1 Voltage	2.68 V
Cell 2 Voltage	2.59 V
Cell 3 Voltage	2.64 V
Cell 4 Voltage	2.76 V
Pack Voltage	10.67 V
Average Voltage	2.67 V
Maximum Voltage	2.76 V
Minimum Voltage	2.59 V
Imbalance Voltage	0.17 V
Imbalance Percentage	6.35 %
Strongest Cell	Cell 4
Weakest Cell	Cell 2
Battery Health	CRITICAL IMBALANCE

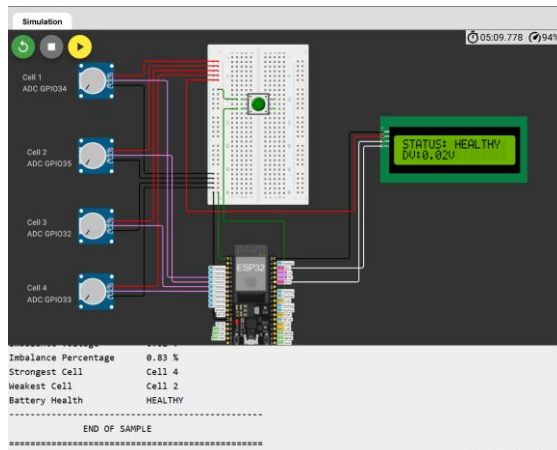
END OF SAMPLE

Battery Health = Pack Failure



ADAPTIVE BATTERY INTELLIGENCE	
Parameter	Value
Cell 1 Voltage	3.04 V
Cell 2 Voltage	2.83 V
Cell 3 Voltage	2.64 V
Cell 4 Voltage	2.76 V
Pack Voltage	11.26 V
Average Voltage	2.82 V
Maximum Voltage	3.04 V
Minimum Voltage	2.64 V
Imbalance Voltage	0.40 V
Imbalance Percentage	14.34 %
Strongest Cell	Cell 1
Weakest Cell	Cell 3
Battery Health	PACK FAILURE

Battery Health = Pack Failure



ADAPTIVE BATTERY INTELLIGENCE	
Parameter	Value
Cell 1 Voltage	3.00 V
Cell 2 Voltage	2.99 V
Cell 3 Voltage	2.99 V
Cell 4 Voltage	3.02 V
Pack Voltage	12.00 V
Average Voltage	3.00 V
Maximum Voltage	3.02 V
Minimum Voltage	2.99 V
Imbalance Voltage	0.02 V
Imbalance Percentage	0.83 %
Strongest Cell	Cell 4
Weakest Cell	Cell 2
Battery Health	HEALTHY

2.8 Important Notes:

[Threshold Limits]

Imbalance % / Condition	Health State	LED Indication
0 – <3%	HEALTHY	● Green
3 – 10%	MINOR_IMBAL	● Yellow
>10%	CRITICAL_IMBAL	● Orange
Any cell UV	PACK_FAILURE	● Red

2.9 Link of Simulation :

[Task 1 Click Here](#)

2.10 Task 1 Video Demonstration Link:

[Click Here](#)

3. Task 2: Battery Intelligence Engine Safety Kernel Module

3.1 Task 2 Problem

Event-Driven Safety Protection Kernel: This task focuses on developing an automotive-grade safety protection controller using a fully non-blocking event-driven architecture. Candidates must detect conditions such as weak cell voltage, overvoltage, sensor anomalies, and rapid voltage fluctuations while controlling relay cutoffs, buzzer alerts, and LCD warning messages. The implementation must rely entirely on millis()-based timing without using delay() and should include recovery logic, anti-relay chatter protection, and stable state management.

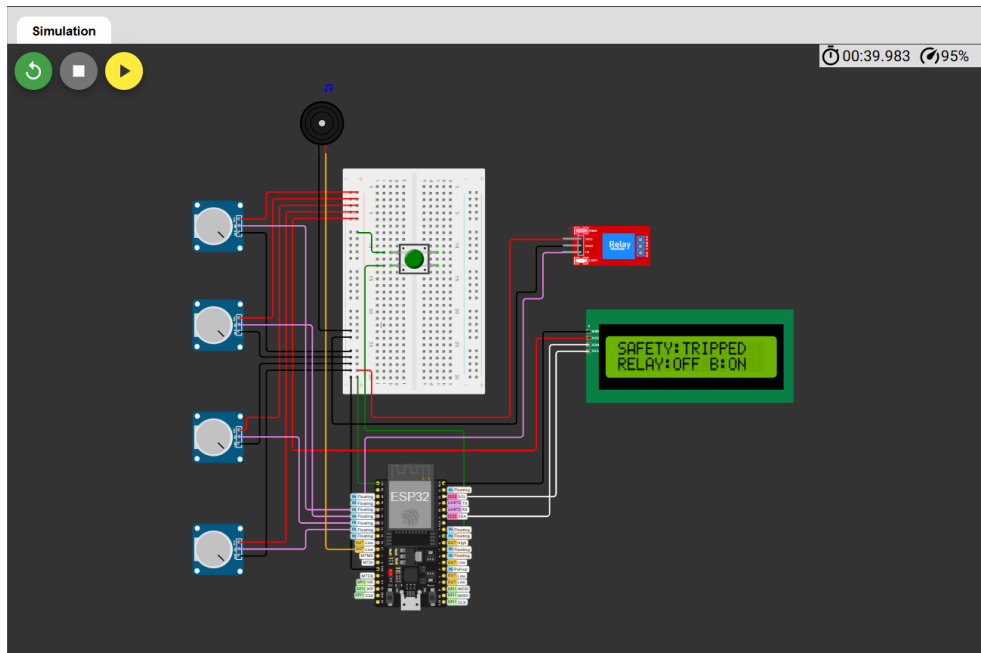
3.2 Components

1. Microcontroller (ESP32- DevKit)
2. Breadboard
3. Pack of Cell (4-Cell) / Potentiometer (10K)
4. LCD 20x4 I²C
5. Push Button

New Components

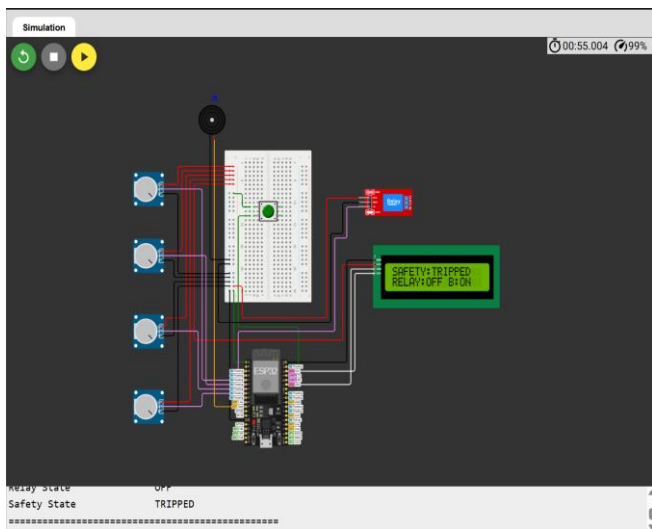
6. Relay Module
7. Buzzer

3.3 Simulation Circuit :



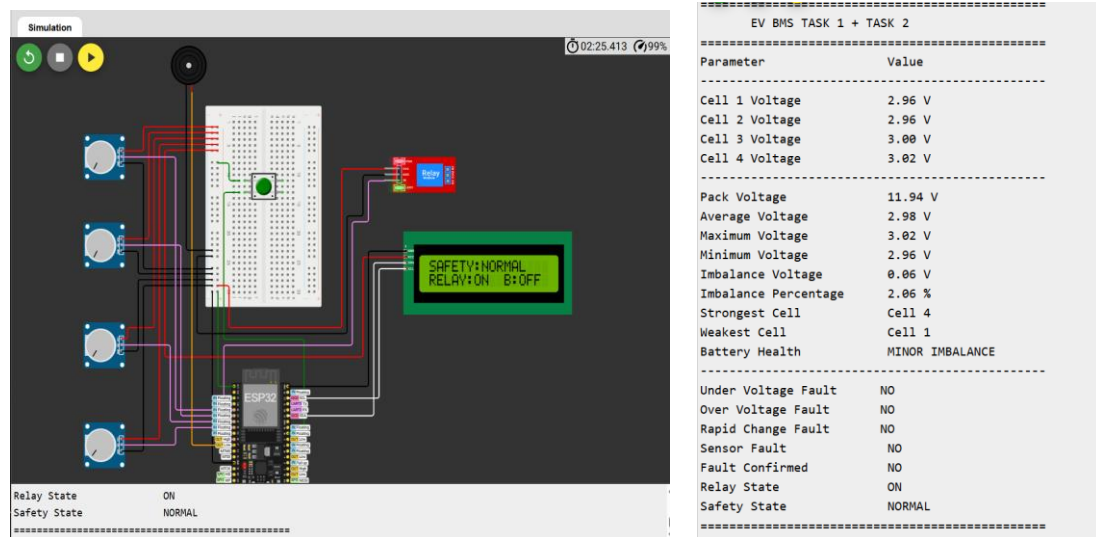
3.4 Simulation Running Photos with correct Output:

[Relay State = off; Safety State = Tripped]



EV BMS TASK 1 + TASK 2	
Parameter	Value
Cell 1 Voltage	2.50 V
Cell 2 Voltage	2.50 V
Cell 3 Voltage	2.50 V
Cell 4 Voltage	2.50 V
Pack Voltage	10.00 V
Average Voltage	2.50 V
Maximum Voltage	2.50 V
Minimum Voltage	2.50 V
Imbalance Voltage	0.00 V
Imbalance Percentage	0.00 %
Strongest Cell	Cell 1
Weakest Cell	Cell 1
Battery Health	HEALTHY
Under Voltage Fault	YES
Over Voltage Fault	NO
Rapid Change Fault	NO
Sensor Fault	NO
Fault Confirmed	YES
Relay State	OFF
Safety State	TRIPPED

[Relay State = On; Safety State = Normal]



3.5 Important Notes:

Test	Input Condition	Expected Result
1. Normal	All cells ≈ 3.8 V	NORMAL, Relay ON
2. Under-Voltage	Any cell < 2.8 V	Relay OFF + Buzzer ON
3. Over-Voltage	Any cell > 4.15 V	Relay OFF + Buzzer ON
4. Rapid Change	Voltage change > 0.10 V/sample	Fault detected
5. Sensor Fault	Invalid/out-of-range reading	Safe state + Warning
6. Recovery	Restore normal voltage	Recovery $\rightarrow$ Normal $\rightarrow$ Relay ON
7. Relay Chatter	Repeated fault/normal transition	Relay switching restricted

3.6 Link of Simulation (Wokwi):

[Task 2 Click Here](#)

3.7 Task 2 Video Demonstration Link:

[Click Here](#)

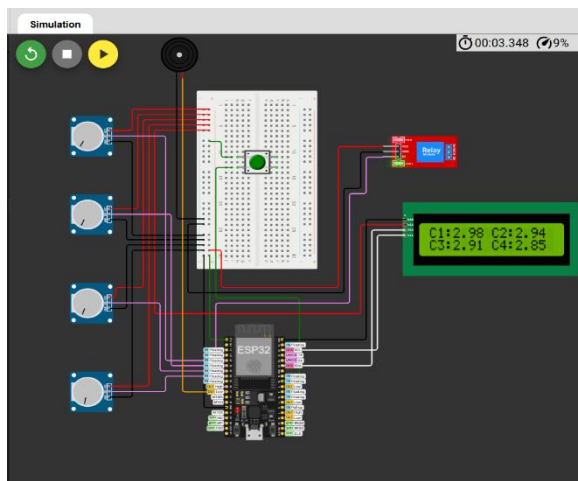
4. Task 3 : Intelligent Embedded HMI & Diagnostic Interface

4.1 Task 3 Problem

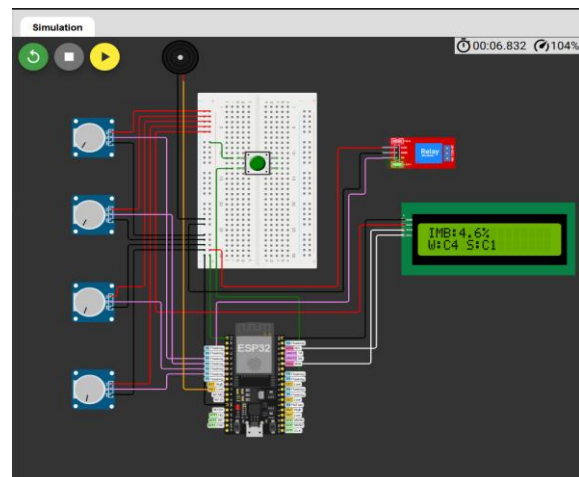
Intelligent Embedded HMI & Diagnostic Interface: In this task, participants must build a professional Human-Machine Interface (HMI) for the embedded system using an LCD display. The interface should automatically rotate between multiple diagnostic screens displaying live battery data, analytics, protection status, and fault diagnostics. The system must support smooth, flicker-free rendering, optimized LCD refresh handling, and automatic fault-priority screen overrides during critical conditions.

4.2 Simulation Running Photos with correct Output:

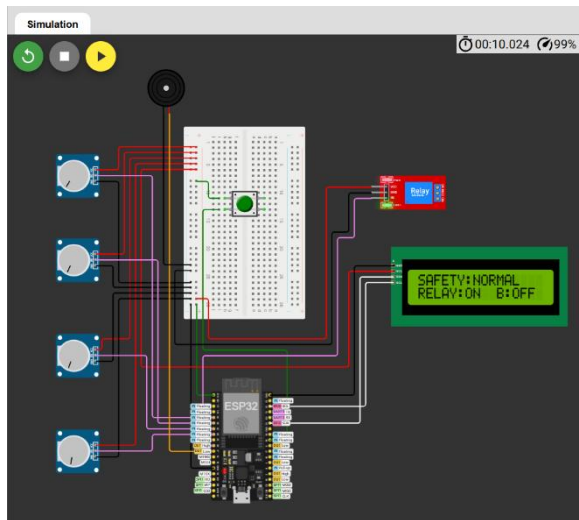
1. Live Cell Data



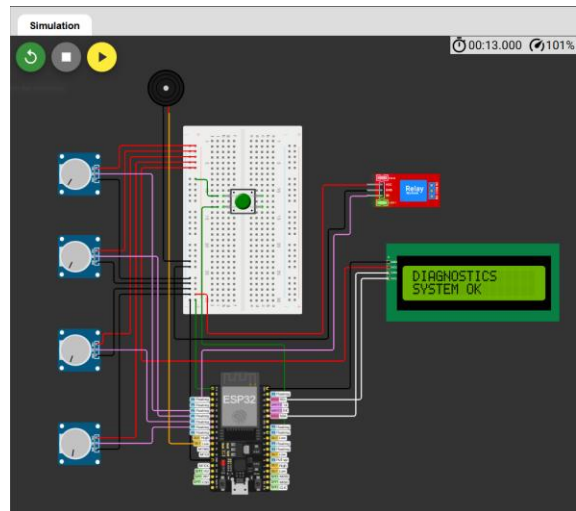
2. Cell Analytics



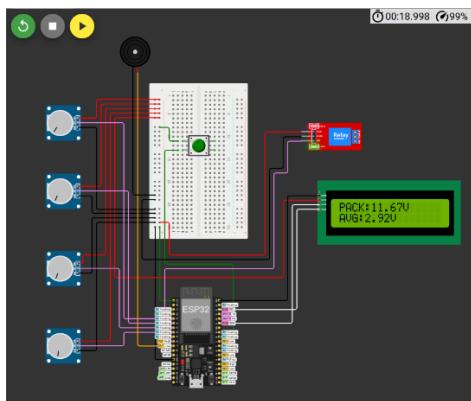
3. Protection :



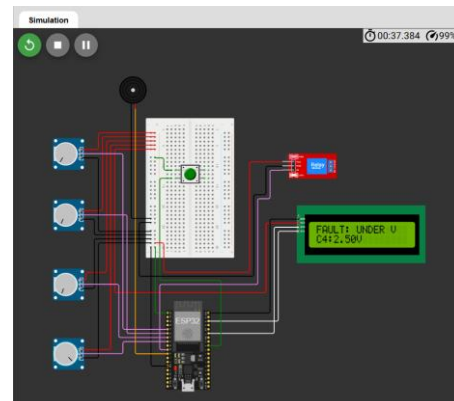
4. Diagnostics :



5. Battery Analytics



6. Fault Comes



4.3 Task 3 Wokwi Link :

[Task 3 Click Here](#)

4.4 Task 3 Video Demonstration Link:

[Click Here](#)

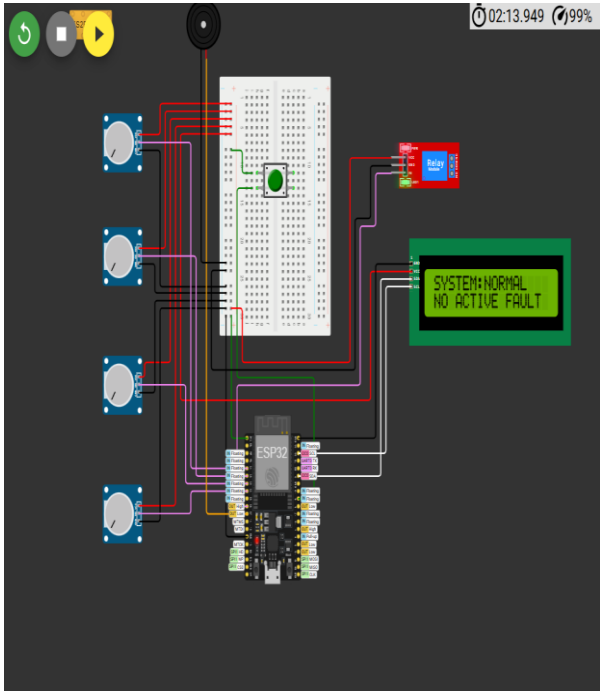
5. Task 4 : Fault-Tolerant Embedded Runtime System

5.1 Task 4 Problem

Fault-Tolerant Embedded Runtime System: This task requires candidates to create a fault-tolerant runtime subsystem capable of detecting and handling hardware and software failures autonomously. The implementation should identify issues such as sensor disconnection, invalid readings, frozen ADC conditions, and relay mismatches while transitioning between operational modes like NORMAL, DEGRADED, FAILSAFE, and SHUTDOWN. The system must isolate faults without affecting healthy modules and maintain timestamped fault logs with structured recovery mechanisms.

5.2 Simulation Running Photos with correct Output:

Case 1 — Normal



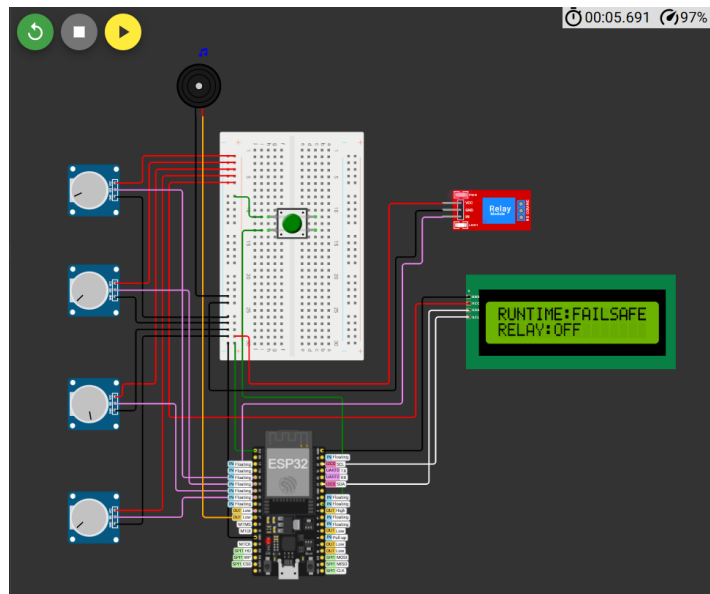
```
=====
EV BMS TASK 1 + 2 + 3 + 4
=====
BATTERY PARAMETERS
-----
Cell 1 Voltage      3.00 V
Cell 2 Voltage      3.00 V
Cell 3 Voltage      2.99 V
Cell 4 Voltage      3.01 V
Pack Voltage        12.00 V
Average Voltage      3.00 V
Imbalance            0.61 %
Strongest Cell       C4
Weakest Cell         C3
Battery Health       HEALTHY

SAFETY STATUS
-----
Under Voltage       NO
Over Voltage        NO
Rapid Voltage Change NO
Safety State        NORMAL
Relay                ON

FAULT-TOLERANT RUNTIME
-----
Runtime Mode        NORMAL
Invalid Sensor       NO
Frozen ADC           NO
Relay Mismatch       NO

FAULT LOG
-----
No faults logged
=====
```

Case 2 — FAILSAFE / Relay Mismatch



EV BMS TASK 1 + 2 + 3 + 4	
BATTERY PARAMETERS	
Cell 1 Voltage	3.19 V
Cell 2 Voltage	3.07 V
Cell 3 Voltage	2.72 V
Cell 4 Voltage	3.08 V
Pack Voltage	12.07 V
Average Voltage	3.02 V
Imbalance	15.65 %
Strongest Cell	C1
Weakest Cell	C3
Battery Health	CRITICAL IMBALANCE
SAFETY STATUS	
Under Voltage	YES
Over Voltage	NO
Rapid Voltage Change	NO
Safety State	TRIPPED
Relay	OFF
FAULT-TOLERANT RUNTIME	
Runtime Mode	FAILSAFE
Invalid Sensor	NO
Frozen ADC	NO
Relay Mismatch	NO
FAULT LOG	
#1	14290 ms CRITICAL SAFETY FAULT [FAILSAFE]
#2	14302 ms UNDER VOLTAGE [FAILSAFE]
#3	14313 ms RAPID VOLTAGE CHANGE [DEGRADED]

5.3 Task 4 (Wokwi):

[Task-4 Click Here](#)

5.4 Task 4 Video Demonstration Link

[Click Here](#)

6. Task 5 : Intelligent Cloud Telemetry Architecture:

6.1 Task 5 Problem

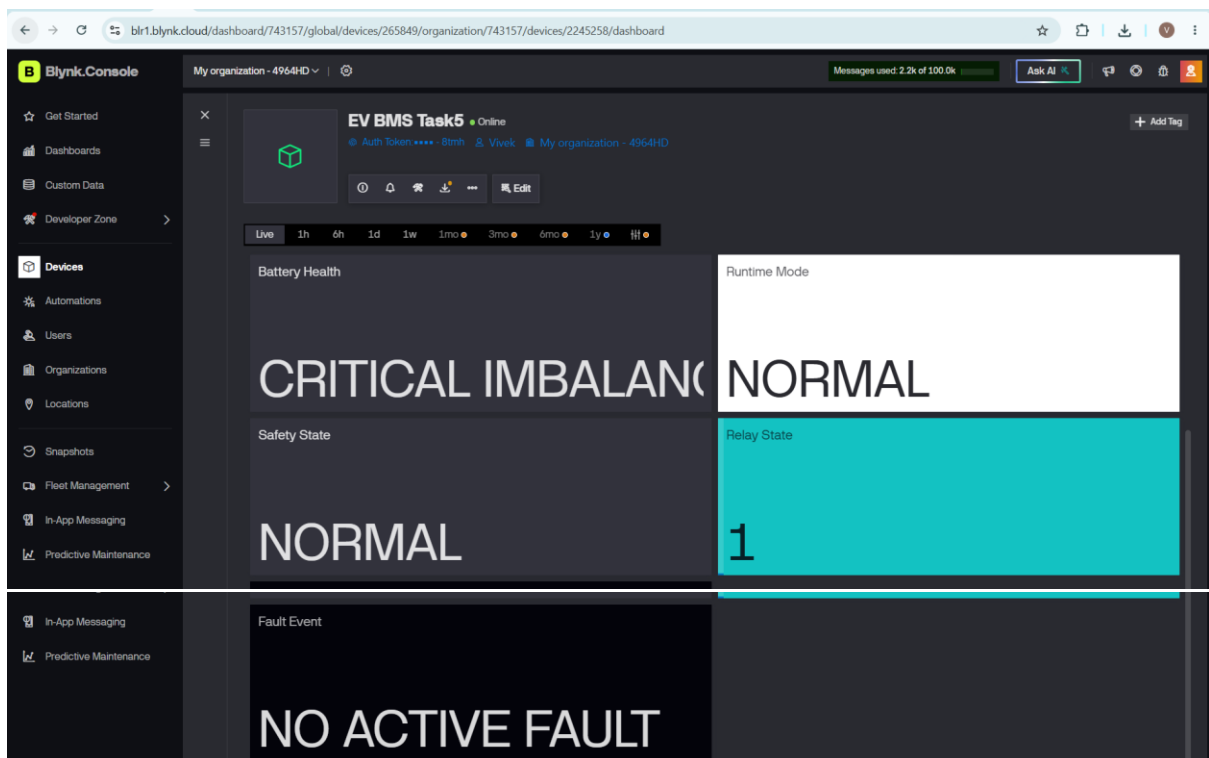
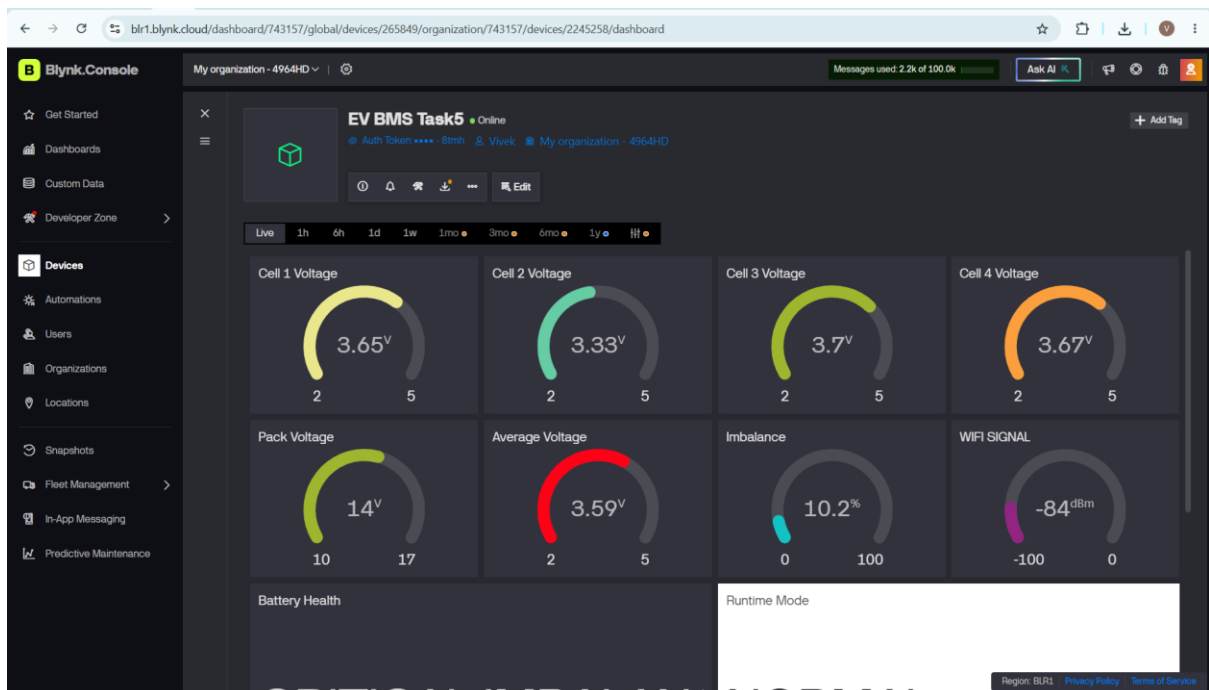
In this task, candidates must design a smart IoT telemetry system using the Blynk platform for cloud communication. Instead of fixed-interval telemetry, the system should transmit data only during state changes, anomalies, or voltage threshold violations. The implementation must include WiFi reconnect handling, event queue synchronization after reconnection, signal quality monitoring, and asynchronous cloud communication while ensuring the embedded system continues functioning normally during network failures.

6.2 Simulation Running Photos with correct Output Of Blynk Dashboard:

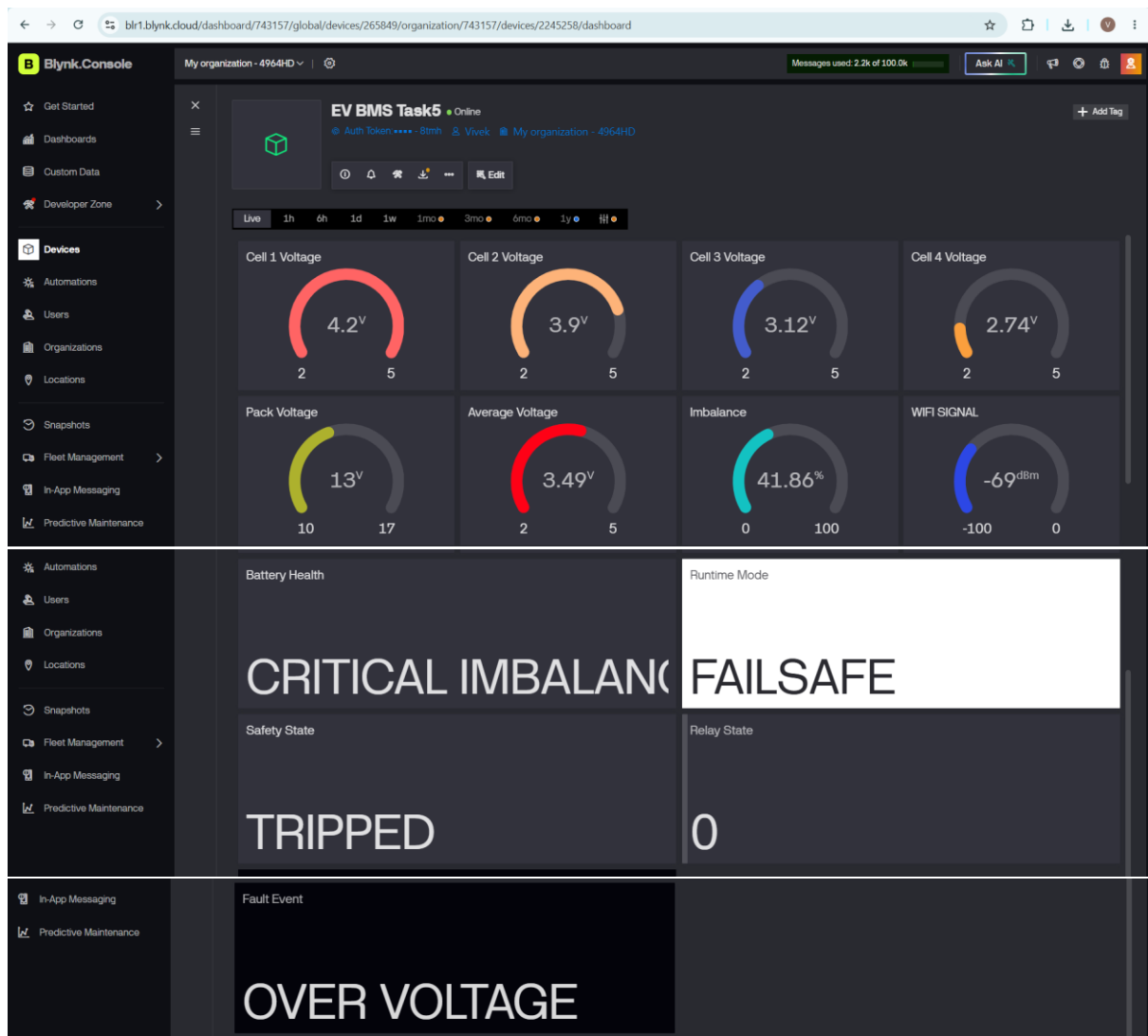
1) Normal BMS and Blynk operation:

The top screenshot shows the Wokwi IDE interface for a project titled "Task-5 Intelligent Cloud Telemetry Architecture Copy". The code in the sketch.ino file includes a loop that updates cell voltages, ADC values, and frozen counts, and prints serial output. The simulation output shows the system starting up, connecting to WiFi, and then reporting a "FAULT EVENT" (CRITICAL SAFETY FAULT) at 3026 ms.

The bottom screenshot shows the same project after a "FAULT-TOLERANT RUNTIME" period. The simulation output shows the system's status after the fault, including a "FAULT LOG" and "CLOUD TELEMETRY" status. The fault log shows the sequence of events: #1 3025 ms CRITICAL SAFETY FAULT [FAILSAFE], #2 3027 ms INVALID SENSOR [DEGRADED], #3 3038 ms UNDER VOLTAGE [FAILSAFE], #4 22426 ms RAPID VOLTAGE CHANGE [DEGRADED], and #5 26454 ms DEGRADED RECOVERED [NORMAL]. The cloud telemetry status shows the system is connected to WiFi and Blynk, with a signal strength of -73 dBm and 2 queued cloud events.



When I changed the values:



2) Blynk communication failure while Wi-Fi remains connected:

WOKWI Task-5 Intelligent Cloud Telemetry Architecture Copy

sketch.ino • diagram.json • libraries.txt • Library Manager

```

692 {
790   for (int i = 0; i < NUM_CELLS; i++)
791   {
792     previousCellVoltage[i] = cellVoltage[i];
793     previousADC[i] = cellADC[i];
794     frozenCount[i] = 0;
795     cellSensorHealthy[i] = true;
796   }
797   updateLEDs();
798   displayNormalLCD();
799   Serial.println();
800   Serial.println("=====");
801   Serial.println("EV BMS TASK 1 + 2 + 3 + 4 READY");
802   Serial.println("=====");
803 }
804
805 // =====
806 // MAIN LOOP
807 // =====
808 void loop()
809 {
810   unsigned long now = millis();
811   if (WiFi.status() == WL_CONNECTED)
812   {
813     if (!cloudTestMode && Blynk.connected())
814     {
815
816
817
818
819
820
821
822
823
824
825
826

```

Simulation

01:48.026 50%

CLOUD TELEMETRY

```

-----
WiFi Status      CONNECTED
Blynk Status     OFFLINE
WiFi RSSI        -82 dBm
Queued Cloud Events  0
=====

```

Blynk Console My organization - 4954HD

Messages used: 2.5k of 100.0k

EV BMS Task5 • Online

Auth token: *****-8888 • Vivek • My organization - 4954HD

Live 1h 6h 1d 1w 1mo 3mo 6mo 1y

Cell 1 Voltage 4.2V (Range: 2 to 5)

Cell 2 Voltage 3.9V (Range: 2 to 5)

Cell 3 Voltage 3.12V (Range: 2 to 5)

Cell 4 Voltage 2.74V (Range: 2 to 5)

Pack Voltage 13V (Range: 10 to 17)

Average Voltage 3.49V (Range: 2 to 5)

Imbalance 41.86% (Range: 0 to 100)

WIFI SIGNAL -89dBm (Range: -100 to 0)

Battery Health

Runtime Mode

CRITICAL IMBALANCE FAULTSAFE

Safety State

TRIPPED

Relay State

0

Fault Event

OVER VOLTAGE

3) Blynk reconnection and cloud synchronization:

The image displays two screenshots related to Blynk reconnection and cloud synchronization.

Top Screenshot: Wokwi IDE

The Wokwi IDE interface shows a sketch titled "Task-5 Intelligent Cloud Telemetry Architecture Copy". The code is in C++ and includes a loop that updates cell voltages, ADC values, and frozen counts. It also includes a function to update LEDs and display normal LCD. The code is as follows:

```
692 {
693   for (int i = 0; i < NUM_CELLS; i++)
694   {
695     previousCellVoltage[i] = cellVoltage[i];
696     previousADC[i] = cellADC[i];
697     frozenCount[i] = 0;
698     cellSensorHealthy[i] = true;
699   }
700   updateLEDs();
701   displayNormalLCD();
702   Serial.println();
703   Serial.println("=====");
704   Serial.println("EV BMS TASK 1 + 2 + 3 + 4 READY");
705   Serial.println("=====");
706 }
707
708 // =====
709 // MAIN LOOP
710 // =====
711
712 void loop()
713 {
714   unsigned long now = millis();
715   if (WiFi.status() == WL_CONNECTED)
716   {
717     if (IcloudTestMode && Blynk.connected())
```

The simulation window shows a circuit diagram and a terminal output. The terminal output displays the following information:

```
CLOUD TELEMETRY
-----
WiFi Status      CONNECTED
Blynk Status     CONNECTED
WiFi RSSI        -78 dBm
Queued Cloud Events 0
=====
>>> BLYNK CONNECTED - CLOUD SYNC <<<
=====
EV BMS TASK 1 + 2 + 3 + 4
=====
BATTERY PARAMETERS
-----
Cell 1 Voltage    4.20 V
Cell 2 Voltage    3.90 V
Cell 3 Voltage    3.12 V
Cell 4 Voltage    2.74 V
Pack Voltage      13.96 V
Average Voltage    3.49 V
Imbalance         41.86 %
Strongest Cell    C1
```

Bottom Screenshot: Blynk Console

The Blynk Console interface shows the "EV BMS Task5" device online. The dashboard displays various battery parameters as gauges:

- Cell 1 Voltage: 4.2V
- Cell 2 Voltage: 2.94V
- Cell 3 Voltage: 2.75V
- Cell 4 Voltage: 3.2V
- Pack Voltage: 13V
- Average Voltage: 3.24V
- Imbalance: 45.91%
- WIFI SIGNAL: -86 dBm

The bottom section of the dashboard shows the "Battery Health" status as "CRITICAL IMBALANCE FAILSAFE" and the "Safety State" as "TRIPPED". The "Runtime Mode" is set to "0".



Example 2

WOKWI SAVE SHARE Task-5 Intelligent Cloud Telemetry Architecture Copy Docs

sketch.ino • diagram.json • libraries.txt • Library Manager

Simulation

```

692 {
693   for (int i = 0; i < NUM_CELLS; i++)
694   {
695     previousCellVoltage[i] = cellVoltage[i];
696     previousADC[i] = cellADC[i];
697     frozenCount[i] = 0;
698     cellSensorHealthy[i] = true;
699   }
700
701   updateLEDs();
702   displayNormalLCD();
703   Serial.println();
704   Serial.println("=====");
705   Serial.println("  EV BMS TASK 1 + 2 + 3 + 4 READY");
706   Serial.println("=====");
707 }
708
709 // =====
710 // MAIN LOOP
711 // =====
712
713 void loop()
714 {
715   unsigned long now = millis();
716   if (WiFi.status() == WL_CONNECTED)
717   {
718     if (!cloudTestMode && Blynk.connected())
719     {
720
721
722
723
724
725
726

```

#7 56754 ms RAPID VOLTAGE CHANGE [DEGRADED]
 #8 58373 ms RAPID VOLTAGE CHANGE [DEGRADED]
 #9 58573 ms CRITICAL SAFETY FAULT [FAILSAFE]
 #10 58575 ms OVER VOLTAGE [FAILSAFE]

CLOUD TELEMETRY

WiFi Status CONNECTED
 Blynk Status OFFLINE
 WiFi RSSI -80 dBm
 Queued Cloud Events 0

=====

EV BMS TASK 1 + 2 + 3 + 4

=====

BATTERY PARAMETERS

WOKWI SAVE SHARE Task-5 Intelligent Cloud Telemetry Architecture Copy Docs

sketch.ino • diagram.json • libraries.txt • Library Manager

Simulation

```

692 {
693   for (int i = 0; i < NUM_CELLS; i++)
694   {
695     previousCellVoltage[i] = cellVoltage[i];
696     previousADC[i] = cellADC[i];
697     frozenCount[i] = 0;
698     cellSensorHealthy[i] = true;
699   }
700
701   updateLEDs();
702   displayNormalLCD();
703   Serial.println();
704   Serial.println("=====");
705   Serial.println("  EV BMS TASK 1 + 2 + 3 + 4 READY");
706   Serial.println("=====");
707 }
708
709 // =====
710 // MAIN LOOP
711 // =====
712
713 void loop()
714 {
715   unsigned long now = millis();
716   if (WiFi.status() == WL_CONNECTED)
717   {
718     if (!cloudTestMode && Blynk.connected())
719     {
720
721
722
723
724
725
726

```

#9 58573 ms CRITICAL SAFETY FAULT [FAILSAFE]
 #10 58575 ms OVER VOLTAGE [FAILSAFE]

CLOUD TELEMETRY

WiFi Status CONNECTED
 Blynk Status CONNECTED
 WiFi RSSI -66 dBm
 Queued Cloud Events 0

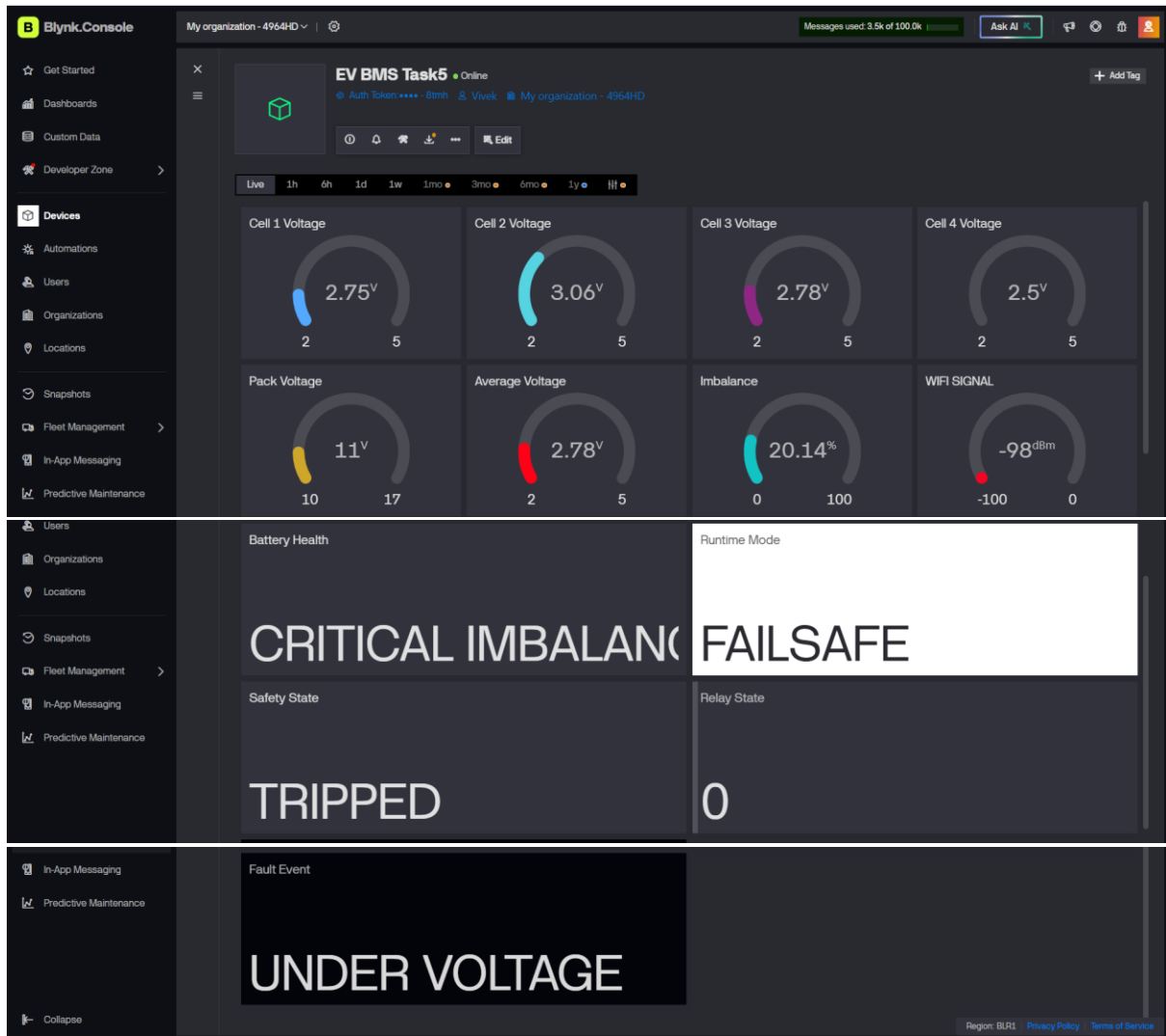
>>> BLYNK CONNECTED - CLOUD SYNC <<<

=====

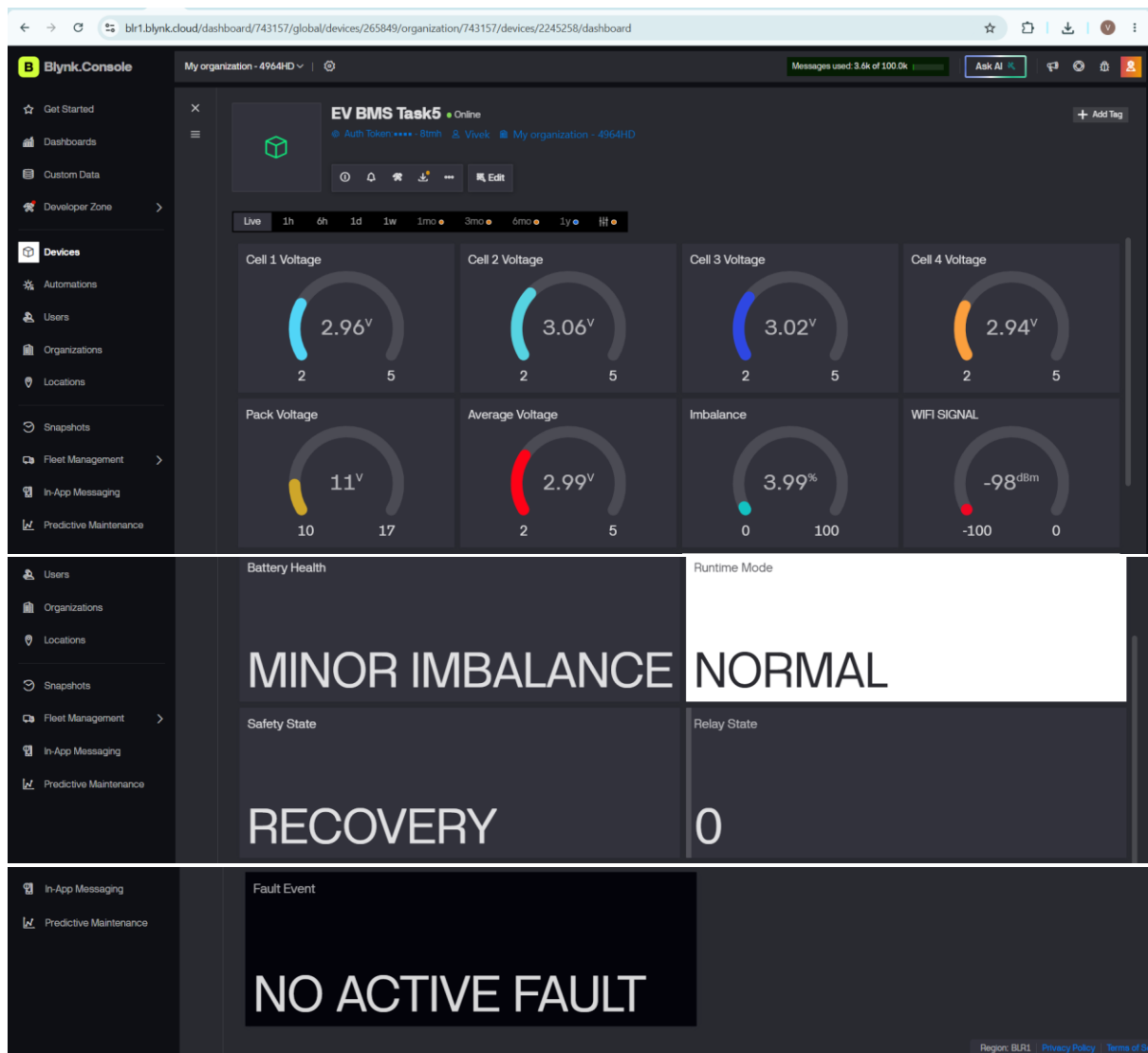
EV BMS TASK 1 + 2 + 3 + 4

=====

BATTERY PARAMETERS



4) Normal BMS operation after cloud recovery:



6.3 Link of Simulation (Wokwi):

[Task 5 Click Here](#)

6.4 Link of Dashboard (Blynk IoT):

[Blynk Dashboard Click Here](#)

6.5 Task 5 Video Demonstration Link:

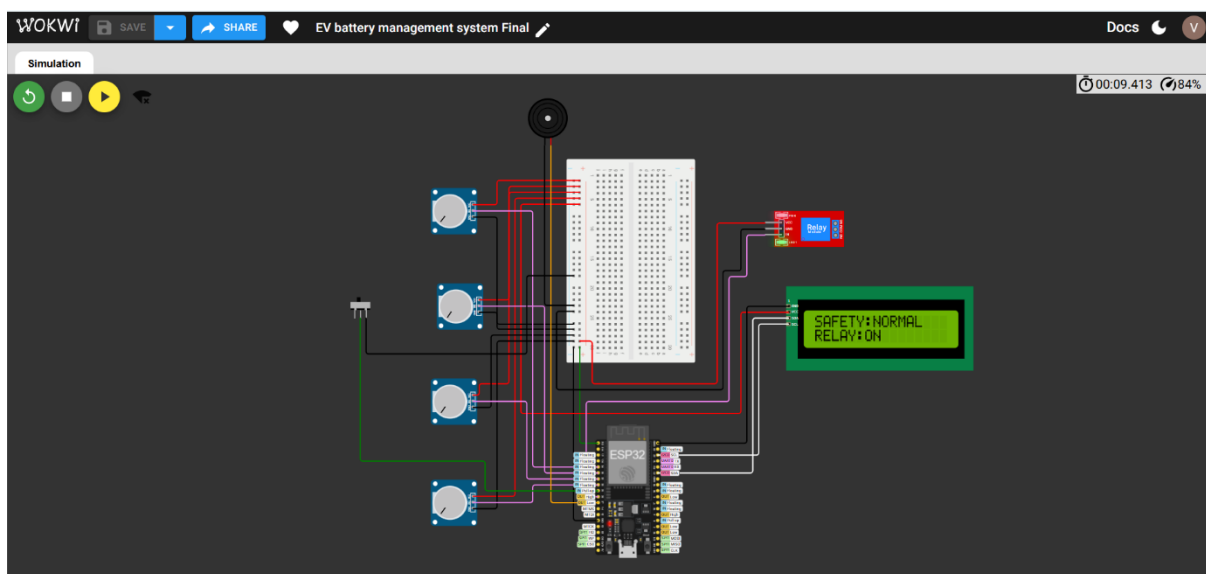
[Click Here](#)

7. Task 6: Executive Battery Intelligence Dashboard

7.1 Task 6 Problem

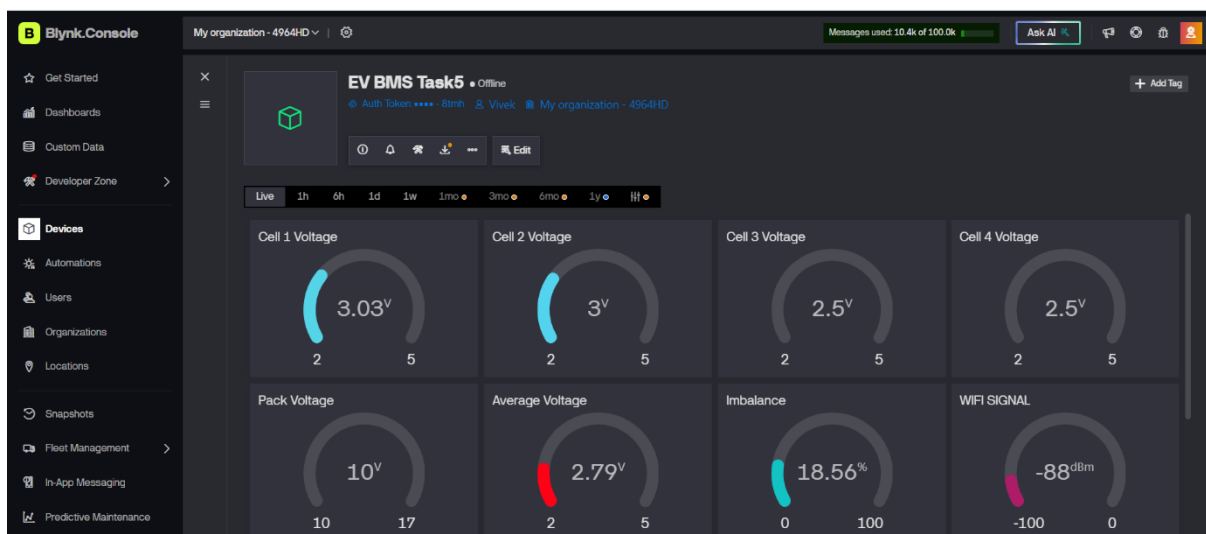
This final task involves creating an enterprise-grade monitoring dashboard that visualizes battery intelligence and operational analytics in a structured and professional manner. The dashboard should include live operational data, risk analysis, historical voltage trends, diagnostics, fault history, and intelligent operator recommendations. It must use severity-based color coding, dynamic telemetry visualization, and actionable insights to provide a professional IoT user experience suitable for real-world battery management systems.

7.2 Final Project Circuit Diagram :

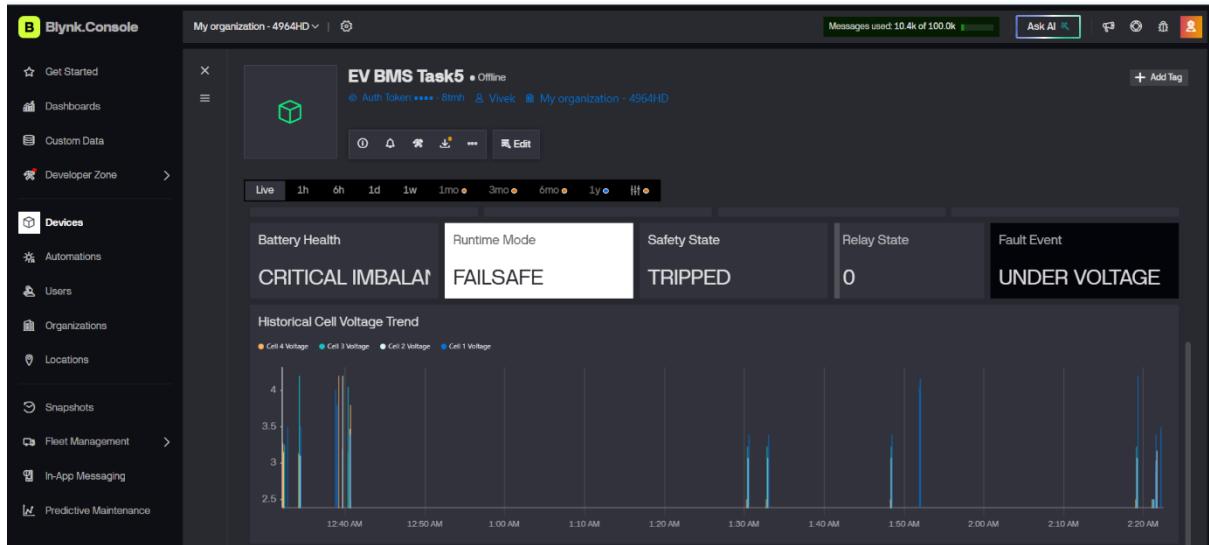


7.3 Blynk Dashboard Overview photos :

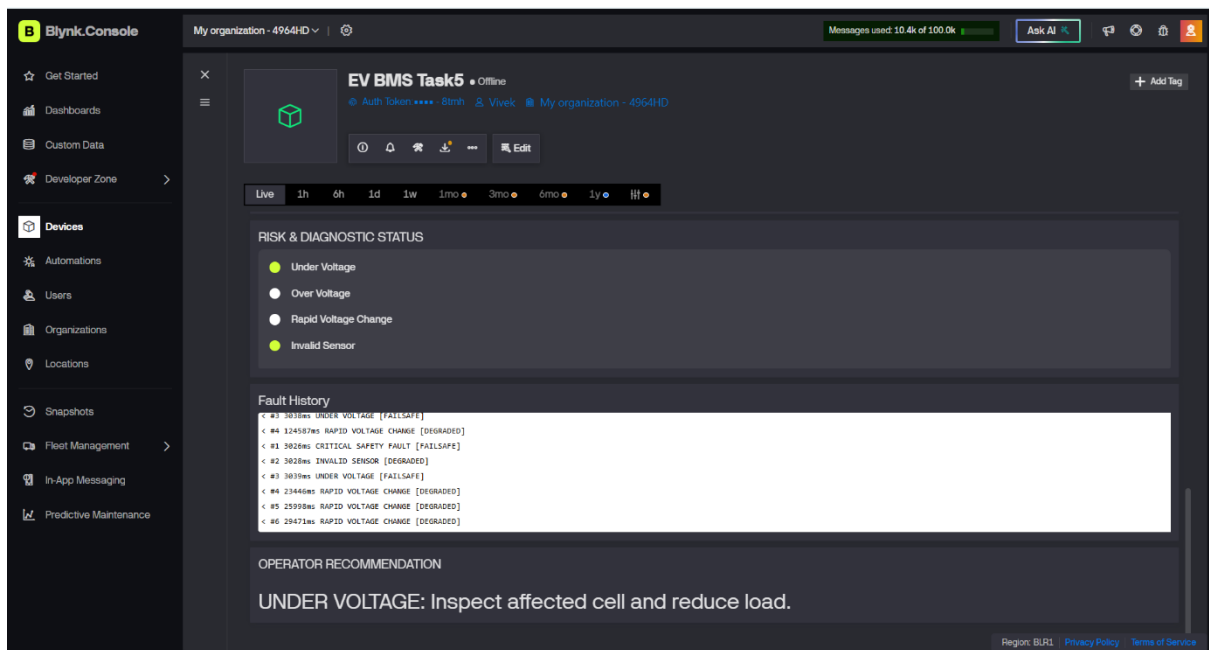
Case-1 Operational data & risk analysis:



Case-2 historical voltage trends:



Case-3 diagnostics, fault history, and intelligent operator recommendations:



7.4 Link of Dashboard :

[Blynk Dashboard Click Here](#)

7.5 Link of Final Wokwi Simulation :

[Task 6 Click Here](#)

7.6 Task 6 Demonstration Video Link:

[Click Here](#)

8. Limitations:

- i. The battery pack and cell voltages are simulated using Wokwi rather than a physical lithium battery pack.
- ii. The prototype mainly focuses on voltage-based battery intelligence and does not include complete physical current and temperature sensing.
- iii. The system has not been validated on an actual EV battery pack or under real automotive operating conditions.
- iv. Cloud communication and dashboard performance depend on Wi-Fi and Blynk connectivity.
- v. The prototype requires further hardware-level and safety validation before practical EV deployment.

9.Future Scope:

1. Implement the system on a real multi-cell lithium battery pack.
2. Add accurate current and temperature sensing.
3. Implement SOC and SOH estimation using advanced algorithms.
4. Add active/passive cell balancing.
5. Integrate CAN communication for automotive applications.
6. Perform Hardware-in-the-Loop (HIL) and real battery testing.
7. Develop an automotive-grade PCB and improve system reliability for real-world deployment.

10. Conclusion:

The Intelligent EV Battery Management System was successfully developed as an integrated ESP32-based platform combining battery intelligence, safety protection, HMI, fault-tolerant runtime, cloud telemetry, and dashboard monitoring. The system demonstrated real-time four-cell monitoring, fault detection, protection response, diagnostic logging, and remote visualization using Wokwi and Blynk. The project provides a strong foundation for further development toward a hardware-based and automotive-oriented Battery Management System.

11. Appendix

Appendix-A : Task-Wise Video Demonstration Links

Task	Video Demonstration
Task 1 – Battery Intelligence	Click Here
Task 2 – Safety Protection	Click Here
Task 3 – HMI & Diagnostics	Click Here
Task 4 – Fault-Tolerant Runtime	Click Here
Task 5 – Cloud Telemetry	Click Here
Task 6 – Executive Dashboard	Click Here

Appendix-B : Project Resource Links

Resource	Link
Final Wokwi Simulation	Click Here
Blynk Dashboard	Click Here
Final Project Demo Video	Click Here

12. References

- 1) BMS Project Training Course
- 2) Datasheet of ESP32 Microcontroller:

[Click Here](#)